■ INTERNSHIP PREP GUIDE

Full-Stack Developer

Interview Preparation — Step by Step

Flutter & Dart	Java	Firebase	Node.js	MySQL	REST API

Prepared for: Abdullah Al Fuad | BSc Computer Science

■ Table of Contents

Each chapter covers: What it is • Why it's used • Key concepts • Code examples • Interview questions

Flutter & Dart

What is Flutter?

Flutter is an open-source UI toolkit made by Google. It lets you build beautiful mobile apps (Android & iOS), web apps, and desktop apps — all from a single codebase. Think of it like this: write the code once, run it everywhere!

What is Dart?

Dart is the programming language used to write Flutter apps. It is easy to learn, especially if you know Java or JavaScript. Google made Dart specifically to power Flutter.

Full name	Flutter SDK (Software Development Kit)
Language	Dart
Made by	Google
Used for	Android, iOS, Web, Desktop apps from 1 codebase
Current version	Flutter 3.x
Main benefit	Fast development, beautiful UI, single codebase

■ Key Concepts (Must Know)

1. Widget — Everything is a Widget

In Flutter, everything on the screen is a Widget — buttons, text, images, padding, columns. There are two types:

- **StatelessWidget** — Does not change. Example: a Label or Icon.
- **StatefulWidget** — Can change/update. Example: a counter button.

```
// Stateless Widget example
class MyLabel extends StatelessWidget {
@override
Widget build(BuildContext context) {
return Text('Hello, Fuad!');
}
}
```

2. setState() — How to Update the UI

Call setState() inside a StatefulWidget to refresh the screen:

```
int counter = 0;

void increment() {
setState(() {
counter++; // UI updates automatically
```

```
});
}
```

3. Common Layout Widgets

Widget	Purpose
Column	Arrange children top-to-bottom (vertical)
Row	Arrange children left-to-right (horizontal)
Container	Box with padding, color, border, size
Text	Show text on screen
ElevatedButton	Clickable button with shadow
Image	Show an image from assets or internet
ListView	Scrollable list of items
Scaffold	Basic page layout with AppBar, Body, FAB

4. Dart Language Basics

```
// Variables
String name = 'Fuad';
int age = 21;
double gpa = 3.8;
bool isStudent = true;

// Function
String greet(String name) {
return 'Hello, \$name!'; // \$ inserts variable
}

// List (Array)
List<String> skills = ['Flutter', 'Dart', 'Firebase'];

// For loop
for (var skill in skills) {
print(skill);
}

// Null safety (Dart 2.12+)
String? nickname; // ? means it can be null
```

■ Common Interview Questions

■ Interview Q: What is the difference between StatelessWidget and StatefulWidget?

StatelessWidget never changes after it's built. StatefulWidget has a State object and can call setState() to rebuild when data changes.

■ Interview Q: What is the Widget tree in Flutter?

Flutter apps are a tree of Widgets nested inside each other — like a family tree. The parent passes data to children. The root is usually MaterialApp > Scaffold > body.

■ Interview Q: What is pubspec.yaml?

It is the configuration file for a Flutter project. You add dependencies (packages), assets (images, fonts), and the app name here. Like package.json in Node.js.

■ Interview Q: How does Flutter achieve cross-platform support?

Flutter has its own rendering engine (Skia/Impeller) that draws every pixel itself. It does NOT use native UI components — so the UI looks the same on Android and iOS.

■ *Internship Tip: Practice building a simple Login + Home screen in Flutter. That alone shows you understand widgets, navigation, and state management.*

Java

What is Java?

Java is one of the most popular programming languages in the world. It follows the principle: 'Write Once, Run Anywhere' — meaning Java code runs on any device that has a Java Virtual Machine (JVM). Java is used for Android development, backend web servers, and enterprise applications.

Paradigm	Object-Oriented Programming (OOP)
Runs on	JVM (Java Virtual Machine)
Used for	Android apps, backend servers, enterprise software
File extension	.java
Compile cmd	javac MyFile.java → java MyFile

■ Key Concepts (Must Know)

1. OOP — 4 Pillars (Most Important!)

Pillar	Simple Explanation	Real Example
Encapsulation	Hide data inside a class using private fields	ATM machine — you press buttons, not internal wires
Inheritance	One class gets properties of another class	Car extends Vehicle
Polymorphism	Same method, different behavior	Animal.speak() → Dog says 'Woof', Cat says 'Meow'
Abstraction	Hide complexity, show only what's needed	TV remote — you don't see circuit, just buttons

2. Class & Object

```
// Class = blueprint
public class Student {
// Fields (attributes)
private String name;
private int age;

// Constructor
public Student(String name, int age) {
this.name = name;
this.age = age;
}

// Method
public void introduce() {
System.out.println("Hi, I am " + name + ", age " + age);
}
}
```

```
// Object = instance of the class
Student fuad = new Student('Fuad', 21);
fuad.introduce(); // Output: Hi, I am Fuad, age 21
```

3. Inheritance

```
public class Animal {
public void speak() { System.out.println('...'); }
}

public class Dog extends Animal {
@Override
public void speak() { System.out.println('Woof!'); }
}

Animal d = new Dog();
d.speak(); // Output: Woof! (Polymorphism!)
```

4. Interface & Abstract Class

- **Abstract class:** Has some implemented methods + some abstract (no body).
- **Interface:** Only method signatures — no body. Like a contract.

```
interface Printable {
void print(); // No body – must be implemented
}

class Report implements Printable {
public void print() {
System.out.println("Printing report...");
}
}
```

5. Exception Handling

```
try {
int result = 10 / 0; // This causes an error!
} catch (ArithmeticException e) {
System.out.println("Error: " + e.getMessage());
} finally {
System.out.println("This always runs.");
}
```

■ Common Interview Questions

■ Interview Q: What is the difference between == and .equals() in Java?

== compares memory address (reference). .equals() compares actual content. Always use .equals() to compare Strings.

■ Interview Q: What is the difference between ArrayList and LinkedList?

ArrayList uses a dynamic array — fast for reading. LinkedList uses nodes — fast for inserting/deleting. For most cases, use ArrayList.

■ Interview Q: What is static in Java?

A static member belongs to the class itself, not to any object. You can access it without creating an object: ClassName.method().

■ Interview Q: What is the difference between abstract class and interface?

Abstract class can have implemented methods and constructors. Interface (before Java 8) only had method signatures. A class can implement multiple interfaces but extend only one abstract class.

■ *Key: Know the 4 OOP pillars with real examples. Interviewers love this question!*

03 Firebase

What is Firebase?

Firebase is a Backend-as-a-Service (BaaS) platform made by Google. It gives you a ready-made backend — database, authentication, storage, and more — without needing to build a server yourself. It is perfect for mobile and web apps.

Made by	Google
Type	Backend-as-a-Service (BaaS)
Database	Firestore (NoSQL) + Realtime Database
Auth	Email, Google, Phone, Facebook login
Hosting	Free web hosting with HTTPS
Storage	Store images, videos, files in cloud
Works with	Flutter, Android, iOS, React, Node.js

■ Key Firebase Services

1. Firebase Authentication

Lets users sign in to your app easily. You don't need to write login/signup logic yourself.

```
// Flutter — Email & Password Login
import 'package:firebase_auth/firebase_auth.dart';

// Sign Up
await FirebaseAuth.instance.createUserWithEmailAndPassword(
email: 'fuad@example.com',
password: 'secret123',
);

// Sign In
await FirebaseAuth.instance.signInWithEmailAndPassword(
email: 'fuad@example.com',
password: 'secret123',
);

// Sign Out
await FirebaseAuth.instance.signOut();
```

2. Cloud Firestore (Database)

Firestore is a NoSQL cloud database. Data is stored as Documents inside Collections — similar to JSON objects inside folders.

```
// Structure: Collection > Document > Fields
// users > userId123 > { name: 'Fuad', age: 21 }
```

```
// Add a document
await FirebaseFirestore.instance.collection('users').add({
'name': 'Fuad',
'age': 21,
});

// Read documents
QuerySnapshot snap = await FirebaseFirestore.instance
.collection('users').get();
for (var doc in snap.docs) {
print(doc['name']);
}

// Real-time updates (stream)
FirebaseFirestore.instance.collection('users')
.snapshots().listen((event) {
print('Data changed!');
});
```

3. Firebase Storage

Upload and download files like images, PDFs, videos to the cloud.

```
// Upload image
final ref = FirebaseStorage.instance.ref('uploads/photo.jpg');
await ref.putFile(imageFile);

// Get download URL
String url = await ref.getDownloadURL();
```

Firestore vs Realtime Database

Feature	Cloud Firestore	Realtime Database
Data model	Documents + Collections	JSON tree
Queries	Powerful queries	Limited queries
Offline support	Yes	Yes
Recommended?	Yes (modern choice)	Legacy, older projects

■ Common Interview Questions

■ Interview Q: What is the difference between SQL and NoSQL databases?

SQL (like MySQL) stores data in tables with rows and columns — structured. NoSQL (like Firestore) stores data as flexible documents/JSON — unstructured. Firestore is great when data structure changes often.

■ Interview Q: How does Firebase Authentication work?

Firebase Auth creates a user account and returns a JWT (JSON Web Token). This token is sent with every API request to prove who the user is. Firebase verifies the token on the server side automatically.

■ Interview Q: What is a Firestore Stream/Snapshot?

A Stream gives real-time updates. When data in Firestore changes, your app UI updates instantly without refreshing. Use .snapshots() for streams and .get() for one-time reads.

> ■ *Build a simple ToDo app with Firebase — Auth + Firestore. This is the most common Flutter+Firebase internship demo project!*

04 Node.js & Express.js

What is Node.js?

Node.js lets you run JavaScript on the server (backend) — not just in the browser. It is built on Google Chrome's V8 engine and is very fast because it is non-blocking (it doesn't wait for one task to finish before starting another).

What is Express.js?

Express.js is a lightweight web framework for Node.js. It makes building web servers and REST APIs very easy. Think of Node.js as the engine and Express.js as the car body.

Node.js	JavaScript runtime — runs JS on server
Express.js	Web framework on top of Node.js
Package manager	npm (Node Package Manager)
Port (default)	3000
Used for	REST APIs, web servers, real-time apps
Non-blocking	Handles many requests at the same time (async)

■ Key Concepts

1. Create a Simple Server

```
// Install: npm install express
const express = require('express');
const app = express();

app.use(express.json()); // Parse JSON body

// Route — GET request
app.get('/', (req, res) => {
res.send('Hello from Express!');
});

app.listen(3000, () => {
console.log('Server running on port 3000');
});
```

2. HTTP Methods (REST Verbs)

Method	Purpose	Example
GET	Read data	GET /users — get all users
POST	Create new data	POST /users — add new user

PUT	Update all fields	PUT /users/1 — update user id=1
PATCH	Update some fields	PATCH /users/1 — update only name
DELETE	Delete data	DELETE /users/1 — remove user id=1

3. Full CRUD API Example

```
const express = require('express');
const app = express();
app.use(express.json());

let users = [{ id: 1, name: 'Fuad' }];

// GET — Read all users
app.get('/users', (req, res) => {
res.json(users);
});

// POST — Create user
app.post('/users', (req, res) => {
const newUser = { id: Date.now(), ...req.body };
users.push(newUser);
res.status(201).json(newUser);
});

// PUT — Update user
app.put('/users/:id', (req, res) => {
const idx = users.findIndex(u => u.id == req.params.id);
users[idx] = { id: req.params.id, ...req.body };
res.json(users[idx]);
});

// DELETE — Remove user
app.delete('/users/:id', (req, res) => {
users = users.filter(u => u.id != req.params.id);
res.json({ message: 'Deleted' });
});

app.listen(3000);
```

4. Middleware

Middleware is a function that runs between the request and the response. It can log requests, check authentication, parse data, etc.

```
// Custom middleware — logs every request
app.use((req, res, next) => {
console.log(req.method + ' ' + req.url);
next(); // Must call next() to continue!
});

// Auth middleware example
function checkAuth(req, res, next) {
if (!req.headers.authorization) {
```

```
return res.status(401).json({ error: 'Unauthorized' });
}
next();
}

app.get('/secret', checkAuth, (req, res) => {
res.json({ data: 'Secret data here' });
});
```

■ Common Interview Questions

■ Interview Q: What is the Event Loop in Node.js?

Node.js is single-threaded but handles many requests via the Event Loop. When a slow operation (like reading a file) runs, Node doesn't wait — it continues other tasks and comes back when the operation is done. This makes Node.js very fast and scalable.

■ Interview Q: What is the difference between async/await and callbacks?

Callbacks are functions passed to another function, run when done — can get messy (callback hell). Async/await is modern syntax that makes asynchronous code look like synchronous, much cleaner and easier to read.

■ Interview Q: What is req.params vs req.body vs req.query?

- **req.params** — URL parameters: /users/:id → req.params.id
- **req.body** — Data sent in POST/PUT request body (JSON)
- **req.query** — URL query string: /search?name=fuad → req.query.name

■ *You already built a full Flutter + Node.js + MySQL app — that's excellent internship experience! Mention it confidently.*

MySQL

What is MySQL?

MySQL is the world's most popular open-source relational database. Data is stored in tables (like Excel sheets) with rows and columns. You use SQL (Structured Query Language) to talk to the database — to read, add, update, or delete data.

Type	Relational Database Management System (RDBMS)
Language	SQL (Structured Query Language)
Made by	Oracle Corporation (open-source)
Port	3306
Used with	Node.js, PHP, Python, Java, Express.js
Data stored	Tables with Rows and Columns

■ SQL Commands — CRUD Operations

1. Create Table

```
CREATE TABLE users (
id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100) NOT NULL,
email VARCHAR(100) UNIQUE NOT NULL,
password VARCHAR(255) NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

2. INSERT — Add new row

```
INSERT INTO users (name, email, password)
VALUES ('Abdullah Al Fuad', 'fuad@gmail.com', 'hashed_password');
```

3. SELECT — Read data

```
-- Get all users
SELECT * FROM users;

-- Get specific columns
SELECT name, email FROM users;

-- Filter with WHERE
SELECT * FROM users WHERE id = 1;
SELECT * FROM users WHERE email = 'fuad@gmail.com';

-- ORDER and LIMIT
SELECT * FROM users ORDER BY name ASC LIMIT 10;
```

4. UPDATE — Modify data

```
UPDATE users
SET name = 'Fuad Updated', email = 'new@gmail.com'
WHERE id = 1;
```

■■ Always use WHERE in UPDATE/DELETE! Without it, ALL rows are affected.

5. DELETE — Remove data

```
DELETE FROM users WHERE id = 1;
```

6. JOIN — Combine Tables

JOINs let you get data from multiple related tables at once.

```
-- INNER JOIN — rows that match in BOTH tables
SELECT users.name, orders.product
FROM users
INNER JOIN orders ON users.id = orders.user_id;

-- LEFT JOIN — all rows from left + matching from right
SELECT users.name, orders.product
FROM users
LEFT JOIN orders ON users.id = orders.user_id;
```

JOIN Type	Returns
INNER JOIN	Only matching rows from BOTH tables
LEFT JOIN	All rows from LEFT table + matching from right
RIGHT JOIN	All rows from RIGHT table + matching from left
FULL JOIN	All rows from BOTH tables (MySQL: use UNION)

7. Important Keywords

```
-- GROUP BY + COUNT
SELECT country, COUNT(*) as total FROM users GROUP BY country;

-- HAVING (filter after GROUP BY)
SELECT country, COUNT(*) as total FROM users
GROUP BY country HAVING total > 5;

-- LIKE (search pattern)
SELECT * FROM users WHERE name LIKE 'Fu%'; -- starts with Fu
SELECT * FROM users WHERE email LIKE '%gmail.com'; -- ends with

-- IN (multiple values)
SELECT * FROM users WHERE id IN (1, 2, 3);
```

MySQL with Node.js

```
// npm install mysql2
const mysql = require('mysql2');
```

```
const db = mysql.createConnection({
host: 'localhost', user: 'root',
password: '', database: 'myapp'
});

// Query
db.query('SELECT * FROM users', (err, results) => {
if (err) throw err;
console.log(results);
});

// Parameterized query (prevents SQL injection!)
db.query('SELECT * FROM users WHERE id = ?', [userId], (err, results) => {
console.log(results[0]);
});
```

■ Common Interview Questions

■ Interview Q: What is the difference between PRIMARY KEY and FOREIGN KEY?

PRIMARY KEY uniquely identifies each row in a table — no duplicates, no NULL. FOREIGN KEY links a column in one table to the PRIMARY KEY of another table — it creates the relationship between tables.

■ Interview Q: What is SQL injection and how do you prevent it?

SQL injection is when a hacker puts SQL code inside user input to manipulate the database. Prevent it by always using parameterized queries (prepared statements) — never directly concatenate user input into SQL strings.

■ Interview Q: What is the difference between WHERE and HAVING?

WHERE filters rows BEFORE grouping. HAVING filters groups AFTER GROUP BY. You cannot use aggregate functions (COUNT, SUM) in WHERE — use HAVING.

■ *Know the basic CRUD queries and at least one JOIN by heart. Practice with a small project database.*

06 REST API

What is a REST API?

REST stands for Representational State Transfer. An API (Application Programming Interface) is a way for two applications to talk to each other. A REST API is a set of rules for how to communicate over the internet using HTTP.

Example: When your Flutter app needs to get users from the server, it sends an HTTP request to the REST API. The API processes it and sends back data as JSON.

REST means	Representational State Transfer
Protocol	HTTP / HTTPS
Data format	JSON (most common), XML
Stateless	Each request is independent — server doesn't remember previous requests
Client–Server	Frontend (Flutter) asks, Backend (Node.js) answers
Endpoint	A URL that accepts requests: /api/users, /api/login

■ REST API Design Principles

1. HTTP Status Codes — Know These!

Code	Meaning	When to use
200	OK	Request succeeded — data returned
201	Created	New resource created (POST success)
400	Bad Request	Client sent invalid data
401	Unauthorized	Not logged in / no token
403	Forbidden	Logged in but no permission
404	Not Found	Resource does not exist
500	Internal Error	Something crashed on the server

2. RESTful URL Design (Best Practices)

Action	HTTP Method	URL	Good or Bad?
Get all users	GET	/api/users	■ Good
Get one user	GET	/api/users/1	■ Good
Create user	POST	/api/users	■ Good
Update user	PUT	/api/users/1	■ Good

Delete user	DELETE	/api/users/1	■ Good
Get all users	GET	/getUsers	■ Bad (verb in URL)
Delete user	GET	/deleteUser?id=1	■ Bad (GET to delete)

3. JSON Request & Response

```
// Request — POST /api/users
// Headers: Content-Type: application/json
// Body:
{
"name": "Abdullah Al Fuad",
"email": "fuad@example.com",
"password": "secret123"
}

// Response — 201 Created
{
"success": true,
"message": "User created",
"data": {
"id": 42,
"name": "Abdullah Al Fuad",
"email": "fuad@example.com"
}
}
```

4. JWT Authentication in REST APIs

JWT (JSON Web Token) is used to protect API endpoints. After login, the server sends a token to the client. The client sends this token with every future request to prove identity.

```
// 1. Login — server sends JWT
POST /api/login → { token: 'eyJhbGc...' }

// 2. Protected request — client sends token in header
GET /api/profile
Authorization: Bearer eyJhbGc...

// 3. Server verifies token
const jwt = require('jsonwebtoken');
const decoded = jwt.verify(token, process.env.JWT_SECRET);
// decoded = { userId: 42, iat: 123456, exp: 789012 }
```

How Flutter Calls a REST API

```
// Flutter — using http package
import 'package:http/http.dart' as http;
import 'dart:convert';

// GET request
final response = await http.get(
Uri.parse('https://api.example.com/users'),
```

```
headers: {'Authorization': 'Bearer \$token'},
);

if (response.statusCode == 200) {
final data = jsonDecode(response.body);
print(data);
} else {
print('Error: \${response.statusCode}');
}

// POST request
final res = await http.post(
Uri.parse('https://api.example.com/users'),
headers: {'Content-Type': 'application/json'},
body: jsonEncode({'name': 'Fuad', 'email': 'fuad@email.com'}),
);
```

■ Common Interview Questions

■ Interview Q: What does 'stateless' mean in REST?

Each HTTP request must contain all the information needed to process it. The server does not store session data between requests. This is why we send the JWT token with EVERY request — the server doesn't remember you.

■ Interview Q: What is the difference between PUT and PATCH?

PUT replaces the entire resource — you send all fields. PATCH partially updates — you only send the fields you want to change. PATCH is more efficient when updating just one or two fields.

■ Interview Q: What is CORS and why does it cause errors?

CORS (Cross-Origin Resource Sharing) is a security feature in browsers. It blocks requests from a different domain. If your Flutter Web or React app calls your API and gets a CORS error, add this to your Express server:

```
// npm install cors
const cors = require('cors');
app.use(cors()); // Allow all origins (development)

// Or restrict to specific origin (production)
app.use(cors({ origin: 'https://yourapp.com' }));
```

■ Interview Q: How do you secure a REST API?

- Use HTTPS (SSL certificate)
- Use JWT or OAuth for authentication
- Validate all user inputs on the server
- Use parameterized queries to prevent SQL injection
- Rate limiting — limit requests per IP (npm: express-rate-limit)
- Never expose passwords or sensitive data in responses

■ *REST API is the glue that connects your Flutter frontend to your Node.js + MySQL backend. Understanding it well is the most valuable skill for a full-stack internship!*

■ Quick Cheat Sheet — All Topics

Topic	Core Concept	Key Term to Remember
Flutter	Everything is a Widget	setState(), Widget tree, Scaffold
Dart	OOP + null-safety language	\$ string interpolation, async/await
Java	4 OOP Pillars	Encap, Inherit, Poly, Abstraction
Firebase	BaaS — no server needed	Auth, Firestore, Storage, JWT token
Node.js	JS on server, Event Loop	Non-blocking, async, npm
Express.js	Web framework for Node.js	Routes, Middleware, req/res
MySQL	Relational DB — SQL language	CRUD, JOIN, PRIMARY KEY, WHERE
REST API	HTTP request-response pattern	GET/POST/PUT/DELETE, JSON, 200/404

■ Final Tips for the Internship Interview

Talk about your project	You built a full Flutter + Node.js + MySQL app. That's real experience — mention it!
Explain with examples	Don't just define — always give a simple real-world example.
Be honest	If you don't know something, say 'I haven't worked with it deeply yet, but I know the concept...'
Ask questions	At the end, ask about their tech stack or what you'd be working on. Shows interest.
Practice CRUD	Write a GET, POST, PUT, DELETE endpoint from memory. It's the most common test.

Best of luck, Fuad! You've got this. ■